

31/08/26 Level 2 : CORE Linked List

① Reverse Linked List:

problem: Given $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{None}$ reverse
 $40 \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{None}$

The four-step pattern:

- Save next
- Reverse link
- Move prev
- move current

Code:

```
next_node = current.next  
current.next = prev  
prev = current  
current = next_node
```

This 4-line pattern
is the heart of Reverse
Linked List reversal

Python Template:

```
def reverseList(head):  
    prev = None  
    current = head  
    while current:  
        next_node = current.next  
        current.next = prev  
        prev = current  
        current = next_node  
    return prev
```

T.C: $O(n)$
S.C: $O(1)$

Reverse linked list teaches:

- * pointer manipulation
- * Reference preservation
- * In place modification
- * Multiple pointer tracking
- * New head identification

② Find middle of linked list:

Problem: Given $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow \text{None}$.

Brute force:

Traverse $\rightarrow$ count nodes $\rightarrow$ find $n/2 \rightarrow$ traverse again

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$

Calc: $n = 5$ mid_index: $n/2 = 2$

Algorithm:

slow = head
fast = head

while fast and fast.next:

slow = slow.next

fast = fast.next.next

return slow

Python Template:

```
def find_mid(head):
```

```
    slow = head
```

```
    fast = head
```

```
    while fast and fast.next:
```

```
        slow = slow.next
```

```
        fast = fast.next.next
```

```
    return slow
```

T.C: $O(n)$

S.C: $O(1)$

Fast = 2 x speed of Slow } $slow = slow.next$
 $fast = fast.next.next$

③ Fast & Slow pointers / cycle detection in LL

Problem: Given $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$, here 40 next point
determine whether the list contains a cycle. e.p: True

Brute force Approach:

```
def has_cycle(head):
```

```
    visited = set()
```

```
    current = head
```

```
    while current:
```

```
        if current in visited:
```

```
            return True
```

```
        visited.add(current)
```

```
        current = current.next
```

```
    return False
```

T.C: $O(n)$

S.C: $O(n)$

Algorithm: (Floyd cycle - optimal)

```
slow = head
```

```
fast = head
```

```
while fast and fast.next:
```

```
    slow = slow.next
```

```
    fast = fast.next.next
```

```
    if slow == fast:
```

```
        return True
```

```
return False
```

Find mid:

fast reaches end
↓
return slow

cycle detection:

$slow == fast$

↓

cycle exists

Python template:

```
def has_cycle(head):
```

```
    slow = head
```

```
    fast = head
```

```
    while fast and fast.next:
```

```
        slow = slow.next
```

```
        fast = fast.next.next
```

```
    if slow == fast:
```

```
        return True
```

```
    return False
```

T.C : $O(n)$

S.C : $O(1)$

④ Merge Two Sorted Linked Lists:

problem: Given two sorted linked lists

list 1: $1 \rightarrow 3 \rightarrow 5 \rightarrow \text{None}$

list 2: $2 \rightarrow 4 \rightarrow 6 \rightarrow \text{None}$

} $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow \text{None}$

Algorithm:

1. Create dummy node

2. tail = dummy

3. while both lists exists:

 Compare list1 and list2

 Take smaller node

 Attach it to tail

 move corresponding pointer

 move tail

4. Attach whichever list remains

5. Return dummy.next

why Dummy Node?: without dummy node need a special handling to the first node.

Dummy Node = simplifies head management

Python Template:

```
def merge_two_list (list1, list2):
```

```
    dummy = Node(0)
```

```
    tail = dummy
```

```
    while list1 and list2:
```

```
        if list1.data <= list2.data:
```

```
            tail.next = list1
```

```
            list1 = list1.next
```

```
        else:
```

```
            tail.next = list2
```

```
            list2 = list2.next
```

```
    tail = tail.next
```

```
    if list1:
```

```
        tail.next = list1
```

```
    else:
```

```
        tail.next = list2
```

```
    return dummy.next
```

T.C : $O(m+n)$

S.C : $O(1)$

Two sorted sequences

↓

Two pointers

↓

Compare current values

↓

Take smaller

↓

Move that pointer